A Model for Entity Profile Management Through Inference in Trails

André Wagner and Jorge Luis Victória Barbosa

Universidade do Vale do Rio dos Sinos (UNISINOS)
Unidade Acadêmica de Pesquisa e Pós-Graduação
Av. Unisinos, 950, São Leopoldo - RS, Brazil

Abstract

To be written...

1 Introdução

Atualmente, a maioria dos usuários interage com uma grande variedade de sistemas computacionais. Cada vez mais, existe a tendência do aumento da interconectividade entre estes sistemas, devido ao fato da internet estar cada vez mais presente e disponível em qualquer lugar. Assim, pesquisas recentes preveem a integração completa de todos estes sistemas em uma federação de sistemas computacionais, que podem ser descritos como Ambientes Inteligentes [9].

Uma visão para o conceito de ambientes inteligentes foi discutida por Weiser em seu artigo seminal, onde ele apresentou um cenário em que o computador se tornaria invisível e indistinguível de seu ambiente [21]. Neste cenário, a interação entre humanos e computadores ocorreria não como a usual – onde o humano dedica 100% de sua atenção ao computador –, mas constituíra-se de pequenas interações rápidas, geralmente em locais e dispositivos distintos, muitas vezes utilizando a periferia da atenção do usuário [22] ou sem ele se dar conta de estar interagindo com um sistema computacional.

Para que este cenário seja possível, é necessário que os aplicativos tenham condições de considerar o contexto no qual se encontram e adaptar-se a este contexto, ou seja, é necessário que sejam "sensíveis a contexto" [6]. Para que um aplicativo possa se adaptar a um usuário de forma eficaz, é importante que ele conheça não apenas o histórico de contextos visitados por um usuário, mas conheça quem o usuário é. As aplicações deste tipo organizam seu conhecimento a respeito do usuário em perfis de usuário, os quais armazenam as informações em um formato baseado em um modelo de usuário específico [18].

Pesquisas nesta área têm indicado que não apenas é interessante conhecer o contexto do usuário no momento em que um determinado serviço está sendo oferecido, mas que o histórico de contextos visitados pode ser uma informação valiosa na adaptação da aplicação ao usuário. Este histórico de contextos recebe o nome de Trilha [16].

Para que um sistema tenha sucesso em obter a intenção do usuário, é necessário que ele conheça o usuário, o contexto que o usuário está inserido e as ações que ele tomou anteriormente quando se encontrava no mesmo contexto ou em contextos semelhantes. As informações do usuário podem se enquadrar em um Perfil de Usuário, enquanto as informações históricas de contexto se encaixam na ideia de Trilha [14].

Este artigo está organizado da seguinte forma. A Seção 2 descreve os principais conceitos referentes às áreas relacionadas a este trabalho. Na Seção 3 é proposto o modelo do eProfile, bem como sua forma de operação. A Seção 4 apresenta um estudo de caso detalhando a utilização do eProfile. A Seção 5 descreve trabalhos relacionados, apresentando as contribuições deste modelo. Por fim, a Seção 6 apresenta as considerações finais.

2 Conceitos

2.1 Computao Ubqua

Em 1991, uma equipe do Laboratório de Ciência Computacional da *Xerox Palo Alto Research Center*, liderada por Mark Weiser, identificou um conjunto de tendências relacionadas à computação [21]:

- o tamanho dos computadores estava se tornando cada vez menor;
- o custo do hardware estava se tornando cada vez menor – tendência que continua até os dias atuais;
- a proporção entre o número de pessoas e o número de computadores estava se tornando cada vez maior (em 1960, era comum o uso de um único computador por centenas de pessoas – atualmente, é comum que cada pessoa possua vários dispositivos computacionais, como computadores, GPS, telefones celulares, entre outros);
- o acesso a redes *wireless* estava cada vez mais disponível aos usuários. Embora na época do artigo o acesso *wireless* à Internet ainda não fosse uma realidade para a maioria dos usuários, é evidente que hoje esta também é uma tendência.

Concluiu-se que todas estas tendências ocasionariam um aumento no número de dispositivos e um aumento da disponibilidade de acesso, que eventualmente levaria a uma sobrecarga de informação sobre os usuários, de forma a causar um detrimento na qualidade da interação entre humanos e computadores e, consequentemente, na qualidade de vida das pessoas [22].

Assim, Weiser inicia seu artigo afirmando que "as tecnologias mais profundas são aquelas que desaparecem. Elas se entrelaçam no tecido do dia-a-dia até que se tornem indistinguíveis dele" [21]. Ele defende que o futuro da computação está em integrar o computador ao mundo real, e chama este novo paradigma de Computação Ubíqua. A noção de realidade virtual é apresentada como o oposto absoluto da computação ubíqua. Enquanto a realidade virtual busca trazer o usuário para dentro de um mundo construído pelo computador, a computação ubíqua busca trazer os computadores para dentro de um mundo construído por pessoas.

O conceito de "tecnologia calma" como solução para o problema de sobrecarga de informações sobre o usuário [22]. Este conceito integra-se intimamente com a computação ubíqua. Dentro desta ideia, a transmissão de informações entre o computador e o usuário devem ocorrer na "periferia" da atenção do usuário, movendo-se para o "centro" da atenção em momentos de urgência. Um exemplo é o motor de um carro: quando um motorista dirige um carro, sua atenção está focada na estrada, e ele mal percebe o som do motor. No entanto, se houver algum som diferente no motor (o que pode indicar algum problema), a atenção do usuário passa a focar-se no motor. Da mesma forma, os sistemas devem, em seu curso normal, manter-se na periferia da atenção do usuário, trazendo a atenção para o centro nos momentos em que a atenção é necessária.

Em termos de sistema, a computação móvel pode ser vista como evolução da computação distribuída, e a computação ubíqua (ou pervasiva, como ele chama no artigo) como evolução da computação móvel [14]. Cada uma destas fases teve seus próprios desafios, e que cada nova fase trouxe junto os desafios da fase anterior.

Assim, a computação ubíqua herda os desafios da computação distribuída e móvel, que são: (i) comunicação remota, (ii) tolerância a falhas, (iii) alta disponibilidade, (iv) acesso a informações remotas, (v) segurança distribuída, (vi) redes móveis, (vii) acesso a informações móveis, (viii) aplicações adaptativas, (ix) sistemas sensíveis a energia e (x) sensitividade a localização. A estes problemas de pesquisa, a computação ubíqua acrescenta:

- uso efetivo de espaços inteligentes: à medida que se acrescenta uma infraestrutura computacional aos espaços físicos, torna-se possível não apenas sentir o mundo exterior, mas também controlá-lo. O contrário também é possível, ou seja, a influência do mundo exterior (como localização) sobre um software do usuário;
- invisibilidade: a busca pelo completo desaparecimento da tecnologia computacional da mente do usuário ou, se inatingível, o mínimo de distração possível;
- escalabilidade localizada: a capacidade de um ambiente suportar uma crescente densidade de dispositivos computacionais, possivelmente através da intensificação das interações locais em detrimento das interações distantes;
- mascaramento de condições irregulares: a capacidade de um sistema de se adaptar a uma taxa de penetração de tecnologia ubíqua variável, dependendo da localização.

2.2 Sensibilidade a Contexto

Para que um sistema seja minimamente intrusivo, é necessário que ele seja capaz de reconhecer a situação do usuário e seus arredores, e adapte seu comportamento de acordo com esta informação. Esta informação é chamada de "contexto" e sistemas capazes deste tipo de adaptação são chamados "sensíveis a contexto" [14].

Uma considerável quantidade de pesquisas tem sido feita na tentativa de obter uma definição exata de contexto. Uma das principais dificuldades nesta definição está no fato que, na computação móvel (e, por extensão, ubíqua) o ambiente está sempre mudando [15]. A configuração de hardware (dispositivo, qualidade da rede, conectividade), localização, grupo de pessoas próximas, entre outros, está em constante mudança.

Uma das definições de contexto mais aceitas é a de que

> Contexto é qualquer informação que possa ser usada para caracterizar a situação de uma entidade. Uma entidade é uma pessoa, local ou objeto que seja considerada relevante para a interação entre um usuário e uma aplicação, incluindo o próprio usuário e aplicação [6].

2.3 Trilhas

Estudos recentes indicam que além do contexto atual de um usuário, o histórico de contextos visitados pode permitir uma ampliação na capacidade de um sistema sensível a contextos de se adaptar a um usuário. A este histórico é dado o nome de Trilha [17, 8, 5].

Embora a trilha geralmente esteja vinculada à localização de um usuário, ela possui um conceito mais amplo, que inclui as atividades realizadas, aplicações usadas, conteúdos acessados – entre outros – dentro de um contexto e um período de tempo específico [17].

A pesquisa na área de trilhas é relevante para o campo do desenvolvimento de aplicações ubíquas, uma vez que a utilização de trilhas vai ao encontro do desafio central da área de computação ubíqua, isto é, o desenvolvimento de aplicações transparentes ao usuário.[7]

2.4 Personalizao

Personalização é a disponibilização de produtos, serviços e informações adaptados individualmente. A tecnologia de personalização geralmente envolve programas que aprendam padrões, hábitos e preferências de um usuário [12].

Entidade é uma extensão do conceito de usuário. Enquanto um usuário geralmente refere-se a uma pessoa que está utilizando um aplicativo, a definição de entidade utilizada por estes autores é de que pode ser qualquer objeto móvel, como um carro ou mesmo um dispositivo [17].

Dentro do âmbito da personalização, este trabalho dá à estrutura de dados utilizada para armazenar as informações das entidades por uma aplicação o nome de Modelo de Entidade, enquanto o conjunto de informações referentes a uma entidade específica é nomeada Perfil de Entidade.

A coleta de informações de uma entidade pode ser explícita ou implícita [18]. Na coleta explícita, é requisitado ao usuário que preencha informações através de um formulário pré-definido, enquanto na coleta implícita, a informação é inferida a partir do estudo do comportamento da entidade.

O objetivo principal por detrás dos sistemas de personalização modernos é oferecer às entidades o que elas querem sem requisitar que elas informem isto de forma explícita [12]. Fica claro que este propósito vai ao encontro do propósito da computação ubíqua, isto é, o desenvolvimento de uma tecnologia que "desapareça".

Informações a respeito de uma entidade colhidas por uma aplicação podem ser úteis para outras aplicações utilizadas pela mesma entidade [18]. No entanto, os diferentes modelos de entidade utilizados pelas aplicações tornam difícil a utilização de informações de um mesmo perfil de entidade por várias aplicações. Além disso, algumas das informações da entidade podem ser válidas apenas dentro de um contexto específico, o que dificulta ainda mais que diferentes aplicações compartilharem o mesmo perfil.

Visando resolver estas questões, o conceito de personalização inter-sistemas foi introduzido [13] como forma de compartilhar informações de entidades entre diferentes aplicações. Como o maior problema deste tipo de modelo é o alinhamento entre as diferentes abordagens de personalização, o estudo de sistemas de modelos de usuários (e, por extensão, de entidades, dentro do âmbito deste trabalho) é atualmente o principal desafio da pesquisa na área de personalização [18].

O *IEEE Standard Computer Dictionary* [10] define interoperabilidade como "a capacidade de dois ou mais sistemas ou componentes trocarem informações e utilizarem as informações que foram trocadas". No âmbito da personalização, interoperabilidade é a capacidade de acessar e interpretar informação derivada de múltiplas fontes heterogêneas e integrar esta informação em um modelo de entidade comum. Os dois principais modelos de interoperabilidade são [3]:

- interoperabilidade sintática: onde as diferenças entre os modelos de entidade são resolvidos em nível de aplicação;
- interoperabilidade semântica: onde as diferenças entre os modelos são resolvidos em nível de conhecimento.

As duas principais abordagens utilizadas em sistemas de modelos de entidades interoperáveis são [18]:

- baseados em padrões: são baseados numa visão *t*op-down, definindo um padrão que todas as aplicações envolvidas devem observar;
- baseados em mediação: se baseiam numa visão *b*ottom-up, onde ocorre algum tipo de reconciliação entre os diferentes modelos de representação.

3 Modelo

Pesquisas apontem que é desejável que os sistemas do futuro se tornem cada vez mais invisíveis aos seus usuários [22], e que um sistema só pode se tornar invisível se for próativo, e só pode ser pró-ativo se conhecer o usuário [14]. Para que um sistema possa conhecer uma entidade, ele deve conhecer o histórico da entidade, compreender o contexto

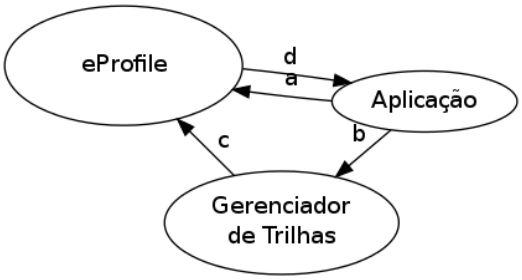

Figura 1: Modelo básico do eProfile

em que ela está inserido, bem como os dados coletados a respeito dela no mesmo contexto (ou num contexto similar), e ser capaz de processar este conjunto de informações em um perfil de entidade.

3.1 Modelo do eProfile

A Figura 1 apresenta a visão geral do eProfile, onde cada círculo representa um componente, e as setas indicam o fluxo dos dados.

O primeiro componente representado é a **aplicação**. A aplicação, dentro deste modelo, é qualquer aplicação que está sendo utilizada diretamente por uma entidade. Para que uma aplicação seja considerada apta a este modelo (isto é, possa utilizar as funcionalidades oferecidas pelo eProfile), ela precisa ser capaz de registrar suas informações em um gerenciador de trilhas suportado pelo eProfile, e deverá seguir o protocolo de comunicação do eProfile para se comunicar com o mesmo. Seguidos estes pré-requisitos, a aplicação pode ser de qualquer tipo e operar em qualquer tipo de dispositivo que tenha comunicação com a internet.

O segundo componente representado é um **gerenciador de trilhas**. Um gerenciador de trilhas é capaz de registrar eventos realizados por uma entidade, colocando-os em uma ordem cronológica e registando informações relevantes ao evento (como o contexto na qual o evento foi realizado). O modelo do eProfile é desenhado para oferecer suporte a qualquer gerenciador de trilhas, desde que o protocolo de comunicação entre o gerenciador de trilhas e o eProfile esteja codificadado no eProfile.

A Figura 1 apresenta ainda um terceiro serviço, o eProfile, que é o modelo apresentado neste trabalho. As setas indicam o fluxo dos dados. A comunicação tem início quando a aplicação registra-se no eProfile, enviando (i) o modelo de entidade utilizado pela aplicação e (ii) as regras para construção deste modelo (seta "a"). A partir deste momento, a aplicação passa a registrar os eventos no gerenciador de trilhas (seta "b"). O eProfile periodicamente busca esta informação a partir do gerenciador de trilhas, e constrói ou reconfigura o perfil de entidade (seta "c"). À medida que a aplicação tem necessidade de um perfil atualizado, ela busca

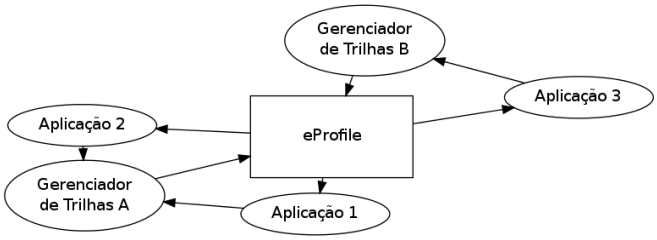

Figura 2: eProfile conectado a mais de um gerenciador de trilhas

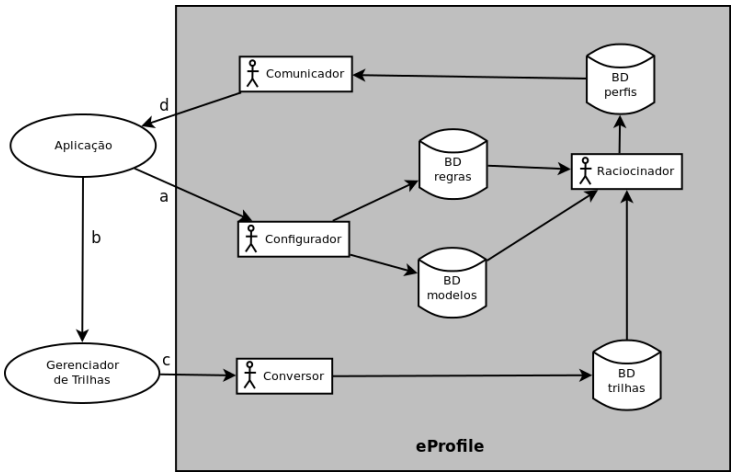

Figura 3: Modelo do eProfile

este perfil no eProfile (seta "d").

Como o eProfile oferece suporte a várias aplicações, é possível que diferentes aplicações utilizem diferentes gerenciadores de trilhas. Por isso, o eProfile oferece também suporte à comunicação com vários gerenciadores de trilhas, segundo exemplificado na Figura 2.

A Figura 3 apresenta o modelo do eProfile. Os serviços "Aplicação" e "Gerenciador de Trilhas" são os mesmos mostrados na Figura 1, enquanto o serviço "eProfile" foi expandido para mostrar seu funcionamento interno.

Cada aplicação tem a possibilidade de montar seu próprio modelo de entidade. Para que o eProfile processe os perfis baseados em um modelo de entidade, ele precisa de três informações: (i) a trilha da entidade, (ii) o modelo de entidade utilizado pela aplicação e (iii) as regras de inferência para o perfil. Isto é visível na Figura 3, onde o agente "Raciocinador" recebe dados destes três bancos. A trilha da entidade é carregada pelo eProfile a partir do gerenciador de trilhas, enquanto as outras duas informações são enviadas diretamente da aplicação para o eProfile. Desta forma, as regras de inferência definidas pela aplicação serão utilizadas para inferir o perfil da entidade.

As setas da Figura 3 indicam a direção do fluxo dos dados, onde podem se verificar dois pontos de entrada e um

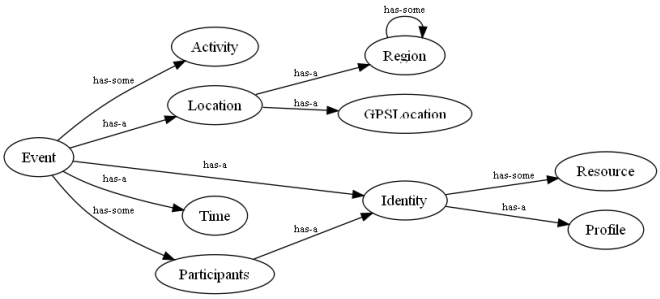

Figura 4: Ontologia de Trilha do eProfile

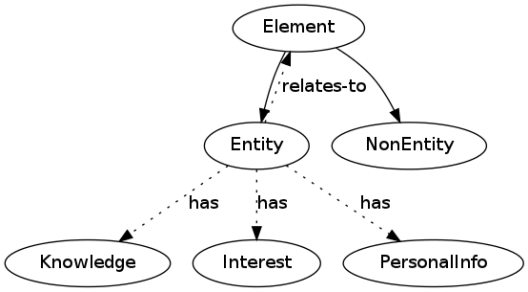

Figura 5: Ontologia de Perfil do eProfile

ponto de saída.

O primeiro ponto de entrada são as informações enviadas pela aplicação ao eProfile ou, mais especificamente, ao agente **configurador**. As informações que devem ser enviadas pela aplicação são as seguintes:

- o modelo de entidade utilizado pela aplicação, ou seja, o formato do perfil da entidade. Uma vez que a trilha da entidade seja processada, as informações inferidas serão armazenadas e disponibilizadas para a aplicação neste formato;

- as regras para inferência do perfil. Estas regras serão executadas em períodos regulares, de modo a atualizar o perfil da entidade. O configurador tem a função de receber as regras, validá-las e inseri-las na **base de regras**. Juntamente com as regras, é especificado a criticidade da informação. O sistema utilizará esta informação para determinar com que frequência a inferência desta informação será executada.

O segundo ponto de entrada é aquele através do qual são recebidas as trilhas das entidades, que são buscadas do gerenciador de trilhas pelo eProfile. Como o eProfile pode se conectar a diferentes gerenciadores de trilha (Figura 2), onde em cada um deles a trilha tem um formato específico, é necessário que seja realizada uma conversão entre o formato original e o formato utilizado pelo eProfile, para que as trilhas sejam armazenadas de forma padronizada. Este trabalho é realizado por um agente intitulado **conversor**, que é capaz de realizar a conversão para os formatos de trilha suportados pelo eProfile.

A trilha da entidade é então armazenada no eProfile em uma ontologia de trilha, chamada **eOntoTrail** (Figura 4). Esta ontologia possui as classes básicas para a construção de uma trilha, sendo que a mesma pode ser estendida de acordo com a especificação do gerenciador de trilhas que está sendo utilizado.

Uma vez de posse das trilhas, do modelo e das regras, o agente **raciocinador** realiza a inferência na trilha, gerando assim um perfil da entidade, que será uma extensão de uma ontologia de perfil básica denominada **eOntoProfile** (Figura 5). Esta ontologia, da mesma forma que a ontologia de trilha, contém classes básicas a partir das quais as aplicações vão especializar, utilizando-as para armazenar informações próprias. A aplicação pode então conectar-se ao eProfile e buscar o perfil de sua entidade, cujos dados são enviados pelo agente **comunicador**.

4 Estudo de Caso

O estudo de caso apresentado é baseado na geração simulada de trilhas de estudantes dentro de um contexto de Educação Ubíqua. Educação Ubíqua pode ser compreendido como processos educacionais que ocorrem em qualquer lugar, a qualquer tempo e com qualquer dispositivo, de forma contínua, contextualizada e integrada ao cotidiano do aprendiz [1].

Foi decidido pela utilização de um cenário simplificado, de modo que o mesmo pudesse ser apresentado de forma aprofundada.

4.1 Definio do Cenrio

O cenário simula um curso de programação em linguagens orientadas a objeto. Deste curso participam três alunos (André, Pedro e Maria), cada um dos quais representado por uma entidade no eProfile.

O curso consiste em um conjunto de módulos relacionados a orientação a objetos. Destes módulos fazem parte conceitos básicos da matéria (classes, herança, *interfaces* e ponteiros) e linguagens de programação orientadas a objeto (C++, Python e Java). O estudante não é obrigado a concluir todos os módulos. No final do curso, no entanto, ele deve escolher a linguagem de programação que ele utilizará para responder as questões da prova final. Dependendo da linguagem que escolher, ele deve ter concluído, no mínimo, os módulos referentes àquela linguagem.

Por exemplo, caso o aluno decida realizar a prova em Java. Neste caso ele deve ter estudado classes, herança e *interfaces*, além do Java propriamente dito. Como a lin-

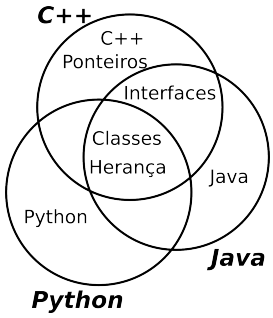

Figura 6: Módulos de estudo obrigatórios

guagem Java não contém o conceito de ponteiros, ele não é obrigado a ter cursado este módulo.

As obrigatoriedades dos módulos são representados pela Figura 6. No exemplo proposto, para que o estudante possa realizar a prova na linguagem Python, ele deve ter concluído no mínimo os módulos Classes, Herança e Python.

O ambiente é integrado a um sistema de educação ubíqua denominado LOCAL [2], utilizado para a distribuição contextualizada de objetos de aprendizagem (LO – *Learning Objects*). Os conteúdos dos módulos são distrubídos pelo LOCAL como objetos de aprendizagem, e o LOCAL registra em um gerenciador de trilhas cada vez que um módulo é iniciado ou concluído.

Cada evento (início ou conclusão de um objeto de aprendizagem) é registrado contendo as seguintes informações: (i) entidade que realizou o evento, (ii) objeto de aprendizagem, (iii) tipo de evento (início ou conclusão), (iii) data e hora e (iv) se é a primeira vez que o estudante realiza este tipo de evento.

O LOCAL registra no gerenciador de trilha não apenas os eventos, mas também as informações relacionadas àqueles eventos. Por exemplo, toda vez que um evento possui relação com um objeto de aprendizagem que está sendo mencionado pela primeira vez, é necessário que sejam incluídas todas as informações relevantes a este objeto, de modo que o eProfile possa utilizar estas informações ao montar a inferência.

Levando em conta o ambiente apresentado, a questão que o LOCAL deseja que o eProfile processe e registre no perfil da entidade é: esta entidade é proeficiente em determinada linguagem? As subsessões a seguir descrevem como ocorre este processo de inferência.

4.2 Ontologia

À medida que o registro das informações vai sendo realizada pelo LOCAL no gerenciador de trilhas, o eProfile vai solicitando informações atualizadas ao gerenciador de trilhas. A forma como o LOCAL e o gerenciador de trilhas se comunicam não é relevante ao eProfile, desde que o gerenciador de trilhas envie as informações em um formato que o eProfile seja capaz de interpretar.

De acordo com a Figura 3, o gerenciador de trilhas envia as informações ao módulo conversor do eProfile, que as converte para o formato OWL. As informações enviadas são de dois tipos: (i) as ocorrências, isto é, a trilha propriamente dita e (ii) as informações acessórias, que ajudarão o eProfile a montar a ontologia necessária para a inferência das informações. No cenário que está sendo descrito, um módulo concluído é um exemplo de ocorrência, enquanto as informações de dependências entre os módulos de aprendizagem são informações acessórias.

Tendo por base as informações enviadas pelo gerenciador de trilhas de acordo com o cenário descrito na Seção 4.1, o eProfile montou a ontologia mostrada na Figura 7. Cada círculo representa uma classe da ontologia.

É importante notar que cada uma das classes pai tem alguma representação na ontologia de trilha do eProfile, apresentada na Figura 4.

Algumas classes possuem informações que são necessárias para que o eProfile possa determinar se um estudante é proeficiente em determinada linguagem. Um exemplo disto são os módulos referentes a conceitos básicos de orientação a objetos, que estão representados na ontologia como classes filhas da classe `ObjectOrientation`. A classe `OO-Interface`, por exemplo, diz respeito a um módulo que estuda o conceito de *interfaces*. Este conceito está presente nas linguagens C++ e Java, mas não em Python. Assim, esta classe é descrita da seguinte forma em OWL[1]:

```
1 Class: <el:OO-Interfaces>
2     SubClassOf:
3         <el:ObjectOrientation>,
4         <el:relatesTo> some <el:C++>,
5         <el:relatesTo> some <el:Java>
```

No exemplo, a linha 1 registra a classe, indicando que ela é filha da classe ObjectOrientation (linha 3, ver Figura 7). Além disso, as linhas 4 e 5 utilizam a propriedade `relatesTo` para indicar que o conceito de *interfaces* está relacionado às linguagens de programação (e, por conseqüência, às classes) `C++` e `Java`. Note-se que não é feita relação à classe `Python`.

Os eventos de requisição e conclusão do estudo de um objeto de aprendizado são registrados como classes filhas da classes `RequestedLOEvent` e `FinishedLOEvent` respectivamente que, por sua vez, são filhas da classe `LearningEvent`. Cada especialização da classe exige informações adicionais, de modo que a classe `FinishedLOEvent`. Este conjunto de classes é representado da seguinte forma:

[1]Embora o eProfile utilize internamente a sintaxe OWL/RDF [20], a sintaxe Manchester-OWL está sendo usada neste artigo por ser mais compacta a amigável [19].

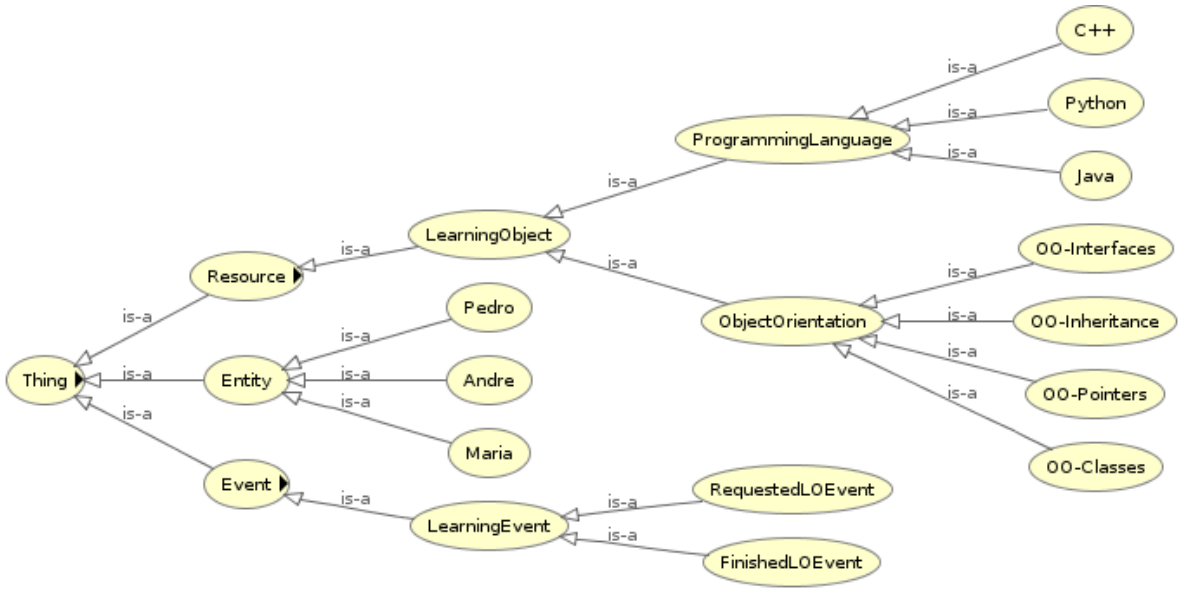

Figura 7: Ontologia do cenário apresentado

```
1  Class: <el:Event>
2      SubClassOf:
3          Thing,
4          <el:hasEntity> some <el:Entity>,
5          <el:hasDateTime> exactly 1 xsd:dateTime,
6  Class: <el:LearningEvent>
7      SubClassOf:
8          <el:Event>,
9          <el:hasLearningObject> some
10                                <el:LearningObject>
11 Class: <el:FinishedLOEvent>
12     SubClassOf:
13         <el:LearningEvent>,
14         <el:isFirstTime> exactly 1 xsd:boolean
```

Desta forma, cada evento registrado deve ser vinculado a uma entidade (linha 4) e ter uma data e hora (linha 5); um evento de aprendizagem deve ter um objeto de aprendizagem relacionado (linhas 9 e 10); e um evento de término de aprendizado deve ter registrado se o evento ocorreu pela primeira vez.

O código abaixo representa o registro de um evento, onde a entidade André (linha 4) concluiu um objeto de aprendizagem sobre Classes (linha 5) pela primeira vez (linha 6) em 08/07/2012 às 10h30 (linha 7). Note-se que, como esta classe é filha da classe `FinishedLOEvent`, as propriedades requisitadas na classe estão sendo informadas nas classes filhas.

```
1  Class: <el:FinishedLOEvent1>
2      SubClassOf:
3          <el:FinishedLOEvent>,
4          <el:hasEntity> some <el:Andre>,
5          <el:hasLearningObject> some <el:OO-Classes>
6          <el:isFirstTime> value true,
7          <el:hasDateTime> value "2012-07-08T10:30:00"
```

4.3 Regras

A aplicação de regras é discutida na Seção **??**.

Para adicionar ao perifl de um aluno se ele é proeficiente em uma determinada linguagem (que é o objetivo do cenário, de acordo com a Seção 4.1, os seguintes passos devem ser realizados:

1. Identificar os objetos de aprendizados requeridos para a proeficiência na linguagem;
2. Identificar os objetos de aprendizagem concluídos pelo estudante referentes ao item 1;
3. Verificar se todos os objetos de aprendizagem necessários foram concluídos.

Neste exemplo, será verificado se o estudante é proeficiente na linguagem Java. Para tal, ele deve ter concluído os objetos de aprendizagens que fazem relação com a linguagem Java (neste caso, classes, herança em *interfaces*), bem como o objeto de aprendizado da linguagem Java. Na ontologia, isto refere-se à criação de uma nova classe (`JavaRequirement`) que seja equivalente à classe `Java` (que é filha da classe de `LearningObject` e, portanto, um objeto de aprendizado, ou quaisquer outras classes cuja propriedade `relatesTo` aponte para a classe `Java`. Assim, esta regra fica definida da seguinte forma:

```
Class: <el:JavaRequirement>
    EquivalentTo:
        <el:Java>
         or (<el:relatesTo> some <el:Java>)
```

O segundo passo é a identificação dos objetos de aprendizagem concluídos pelo estudante relacionados ao Java. Para tal, é necessário que:

1. os eventos sejam do tipo `FinishedLOEvent` (só é importante saber os objetos de aprendizado que o estudante concluiu, e não os que ele iniciou);

2. os eventos tenham sido realizados pela entidade específica;

3. os eventos sejam relacionados ao Java, ou seja, filhas da classe `JavaRequirement` criada acima;

4. os eventos tenham sido realizados pela primeira vez (somente é importante saber os objetos de aprendizado concluídos pelo aluno, e não quantas vezes).

Para que o eProfile possa identificar estes eventos para a entidade Andre, a seguinte regra é enviada ao eProfile:

```
Class: <el:JavaLessonsTaken>
    EquivalentTo:
        <el:FinishedLOEvent>
         and (<el:hasEntity> some <el:Andre>)
         and (<el:hasLearningObject> some
                                <el:JavaRequirement>)
         and (<el:isFirstTime> value true)
```

Uma vez que estas duas regras estejam criadas na ontologia, o eProfile irá executar o raciocinador, que gerará uma nova ontologia OWL, com as classes inferidas identificadas como filhas das classes criadas.

A partir da execução do raciocinador, uma regra de atualização de perfil é criada, onde são comparados o número de módulos Java com o número de objetos de aprendizagem relacionados ao Java que foram concluídos pela entidade. Se o número for igual, o usuário é proeficiente. Esta regra é definida da seguinte forma:

```
JavaProficience <=
  (Count(SubClasses(JavaRequirement))) equals
          (Count(SubClasses(JavaLessonsTaken)))
```

5 Trabalhos Relacionados

Atualmente, as áreas de computação ubíqua, sensibilidade a contexto, trilhas e personalização são bastante promissoras, sendo que um grande número de trabalhos são publicados anualmente a este respeito. No entanto, não foi encontrado nenhum trabalho que faça uso de inferência em trilhas com o objetivo de personalizar a experiência do usuário dentro de um domínio genérico. Assim, o modelo apresentado neste trabalho busca preencher esta lacuna.

O PeLeP [11] suporta aperfeiçoamento automático do perfil de um aprendiz em ambientes de educação ubíqua, utilizando o histórico dos aprendizes para a atualização dos perfis através de inferências. O eProfile utiliza uma estratégia semelhante. No entanto, as regras de inferência são definidas pela aplicação, e não pelo modelo. Esta alteração é necessária para que a inferência passe de um domínio específico (educação) para um domínio genérico. Além disso, ao contrário do eProfile, as regras são dinâmicas (uma aplicação pode definir ou remover regras dinamicamente), ao invés de estarem pré-definidas no modelo. Isto permite a evolução e refino das regras de inferência.

O modelo apresentado por Chellouche et al [4], assim como o eProfile, apresenta um perfil unificado que pode ser utilizado por diversas aplicações. No entanto, ao contrário do eProfile, o modelo assume a interoperabilidade sintática, o que significa que as todas aplicações precisam registrar os eventos utilizando o mesmo formato, o que pode fazer com que a aplicação não consiga registrar o evento com a riqueza de detalhes necessária. Além disso, assim como o PeLeP, o modelo de Chellouche et al [4] está limitado a um domínio específico, que são os serviços multimídia.

O modelo UUCM [13], ao contrário dos outros modelos, trabalha em um domínio genérico. Ele faz isso através da criação de uma ontologia básica que pode ser estendida pelas aplicações. Ao contrário das outras propostas (e do eProfile), o UUCM armazena os perfis no cliente, ao invés do servidor. Isto traz vantagens e desvantagens. Entre as vantagens pode-se citar a privacidade e acesso desconectado. No entanto, esta abordagem limita a interoperabilidade entre sistemas, uma vez que cada cliente fica com uma versão do próprio perfil, restringindo a utilização de um perfil unificado.

A Tabela 1 exibe um comparativo entre o eProfile e os trabalhos relacionados apresentados.

O eProfile fez utilização das ideias apresentadas nos trabalhos, como o aperfeiçoamento automático de perfis, a utilização de perfis unificados e o funcionamento em um domínio genérico. A estas ideias, o eProfile traz como inovação a utilização de trilhas para a extração de perfis, o que facilita a inferência com sensibilidade a contexto. Além disso, o eProfile permite que cada aplicação registre sua trilha no formato que considerar mais proveitoso para si, permitindo assim que a riqueza de detalhes registrados por uma aplicação possa se traduzir em um perfil mais exato da entidade.

Trabalho	Interope-rabilidade	Perfis dinâmicos	Armaze-namento	Sensibilidade a contexto	Utiliza trilhas	Domínio
PeLeP	Não	Sim	Servidor	Sim	Parcial	Educação
Chellouche et al	Sintática	Sim	Servidor	Sim	Não	Serviços multimídia
UUCM	Sintática	Sim	Cliente	Sim	Não	Genérico
eProfile	Semântica	Sim	Servidor	Sim	Sim	Genérico

Tabela 1: Comparação entre os trabalhos apresentados

6 Conclusão

Referências

[1] D. Barbosa, C. Geyer, and J. Barbosa. Globaledu - an architecture to support learning in a pervasive computing environment. In A. Rettberg and C. Bobda, editors, *New Trends and Technologies in Computer-Aided Learning for Computer-Aided Design*, volume 192 of *IFIP International Federation for Information Processing*, pages 1–10. Springer Boston, 2005.

[2] J. Barbosa, R. Hahn, S. Rabello, and D. Barbosa. Local: a model geared towards ubiquitous learning. *SIGCSE Bull.*, 40:432–436, March 2008.

[3] F. Carmagnola and F. Cena. User identification for cross-system personalisation. *Inf. Sci.*, 179:16–32, January 2009.

[4] S. A. Chellouche, J. Arnaud, and D. Négru. Flexible user profile management for context-aware ubiquitous environments. In *Proceedings of the 7th IEEE conference on Consumer communications and networking conference*, CCNC'10, pages 980–984, Piscataway, NJ, USA, 2010. IEEE Press.

[5] S. Clarke and C. Driver. Context-aware trails [mobile computing]. *Computer*, 37(8):97 – 99, aug. 2004.

[6] A. Dey. Understanding and using context. *Personal and Ubiquitous Computing*, 6(1):4–7, 2001.

[7] C. Driver and S. Clarke. Hermes: A software framework for mobile, context-aware trails. In *Workshop on Computer Support for Human Tasks and Activities at Pervasive 2004*, Apr. 2004.

[8] C. Driver and S. Clarke. An application framework for mobile, context-aware trails. *Pervasive Mob. Comput.*, 4:719–736, October 2008.

[9] D. Heckmann. *Ubiquitous User Modeling*. PhD thesis, Department of Computer Science, Saarland University, Germany, November 2005.

[10] IEEE. IEEE standard computer dictionary. a compilation of IEEE standard computer glossaries. *IEEE Std 610*, page 1, 1991.

[11] D. Levis, J. L. V. Barbosa, S. C. C. S. Pinto, and D. N. F. Barbosa. Aperfeiçoamento automático do perfil do aprendiz em ambientes de educação ubíqua. *Revista Brasileira de Informática na Educação*, 16(1), 2008.

[12] M. D. Mulvenna, S. S. Anand, and A. G. Büchner. Personalization on the net using web mining: introduction. *Commun. ACM*, 43:122–125, August 2000.

[13] C. Niederée, A. Stewart, B. Mehta, and M. Hemmje. A Multi-Dimensional, Unified User Model for Cross-System Personalization. In *E4PIA Workshop 2004*, 2004.

[14] M. Satyanarayanan. Pervasive computing: Vision and challenges. *IEEE Personal Communications*, 8:10–17, 2001.

[15] B. Schilit, N. Adams, and R. Want. Context-aware computing applications. In *In Proceedings of the Workshop on Mobile Computing Systems and Applications*, pages 85–90. IEEE Computer Society, 1994.

[16] J. Silva, J. Rosa, J. Barbosa, D. Barbosa, and L. Palazzo. Content distribution in trail-aware environments. *Journal of the Brazilian Computer Society*, 16:163–176, 2010. 10.1007/s13173-010-0015-1.

[17] J. Silva, J. Rosa, J. Barbosa, D. N. F. Barbosa, and L. A. M. Palazzo. Content distribution in trial-aware environments. In *Proceedings of the XV Brazilian Symposium on Multimedia and the Web*, WebMedia '09, pages 15:1–15:8, New York, NY, USA, 2009. ACM.

[18] M. Viviani, N. Bennani, and E. Egyed-Zsigmond. A Survey on User Modeling in Multi-Application Environments. In *The Third International Conference on Advances in Human-oriented and Personalized Mechanisms, Technologies, and Services CENTRIC'10*, pages 111–116. IEEE, Aug. 2010.

[19] W3C. OWL 2 web ontology language manchester syntax, 2009. `http://www.w3.org/TR/owl2-manchester-syntax/`. Acessado em 08/07/2012.

[20] W3C. OWL web ontology language guide, 2011. `http://www.w3.org/TR/owl-guide/`. Acessado em 05/05/2012.

[21] M. Weiser. The computer for the 21st century. *Scientific America*, pages 94–104, 1991.

[22] M. Weiser and J. S. Brown. Designing calm technology. *PowerGrid Journal*, July 1996.